Lab 2 - Reversing Network Protocols

Your Task:

You've been brought in as an outside incident responder. A client is concerned that a bad actor is living inside their network and quietly exfiltrating company secrets. Their internal incident response (IR) team has narrowed the suspicious activity down to a single host and captured two short windows of its network traffic: `net1.pcap` and `net2.pcap`.

The IR team shared one more detail: while they were investigating, they believe the threat actor caught on that his activity was being monitored. He then changed his exfiltration technique to hide what was being delivered in the payload. That second, more careful attempt is the one recorded in `net2.pcap`.

Your job is to review both captures, collect the evidence, recover what was stolen, and then explain what happened in plain language that the client's executives can understand.

Objective

The objective of this lab is to use network forensic tools to uncover secrets transmitted over the network. You will look at the **same** traffic through three different tools, each of which shows it in a different way:

Tool	What it shows you best
Wireshark	Every individual packet and the raw protocol conversation (the "microscope")
NetworkMiner	Hosts, files and credentials automatically carved out of the traffic (the "evidence locker")
ntopng	Flows, volumes and top talkers on a dashboard (the "bird's-eye view")

You will analyze `net1.pcap` with all three tools first, then repeat the process with `net2.pcap`.

> These packet capture files were originally part of CTF challenges I submitted at 2025 Secure Delaware CTF

You are not graded on extracting the secrets. The steps below walk you through it. You are graded on the written findings at the end, where you describe the activity the way you would to a client's leadership team. In the real world, this is one of the most important skills you can build.

Lab Steps

Part 0 - Setup

1. Download both `net1.pcap` and `net2.pcap` files and transfer them to your Kali-VM:

```
scp net*.pcap teddy@172.16.163.128:/home/teddy/
```

> Use your own username and Kali-VM IP address from Lab 1.

2. Create a working folder for each capture's evidence:

```
mkdir -p ~/lab2/net1 ~/lab2/net2
mv ~/net1.pcap ~/lab2/net1/ && mv ~/net2.pcap ~/lab2/net2/
```

3. Install NetworkMiner (runs on Linux through Mono):

```
sudo apt update && sudo apt install -y mono-devel unzip
cd ~ && wget https://www.netresec.com/?download=NetworkMiner -O networkminer.zip
unzip networkminer.zip && cd NetworkMiner_*
chmod +x NetworkMiner.exe
chmod -R go+w AssembledFiles/ Captures/
```

Start it with:

```
mono NetworkMiner.exe --noupdatecheck
```

> Use NetworkMiner 2.8 or newer. These captures use the "Linux cooked capture v2" format, which older versions cannot read.

4. Install Docker:

> NOTE: If your VM is running in `host-only` networking mode, `docker` may not be able to reach their servers, run the following to add DNS records:

```
CON_UUID=$(nmcli -t -f UUID con show --active | head -n 1); nmcli con mod
"$CON_UUID" ipv4.dns "1.1.1.1 8.8.8.8" && nmcli con up "$CON_UUID"
```

```
sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
newgrp docker
```

5. Install the steganography tools you will need for `net2.pcap`:

```
sudo apt install -y steghide python3-pil python3-numpy
```

Part 1 - `net1.pcap`

Wireshark: read the conversation packet by packet

1. In your Kali-VM, open WireShark:

> `Applications > 09 - Discovery > Network Sniffing > wireshark`

2. Open `~/lab2/net1/net1.pcap` (`File > Open`).

3. Get the big picture first. Open `Statistics > Protocol Hierarchy`, then `Statistics > Conversations` (TCP tab).

Write down:

- Which protocols appear, and which one carries most of the data?
- Which IP address and port are involved, and how many separate TCP conversations are there?
- How long is the capture?

4. Filter to web traffic. In the display filter bar type:

```
http
```

You should see a short sequence of requests. For each one note the **method** (`GET` or `POST`), the **URI** and the **time**. Ask yourself: what is the order of events?

5. Follow the upload. Right-click the `POST /upload.php` packet and choose `Follow > HTTP Stream` .

Look for:

- The `Content-Type: multipart/form-data` header (this is a web form file upload)
- The `filename=` value inside the form data
- The server's reply at the bottom of the stream

6. Follow the retrieval. Close the stream window, then follow the `GET /uploads/...` request the same way. What file is requested, and what `Content-Type` does the server return?

7. Carve the files out of the traffic. Go to `File > Export Objects > HTTP` , click **Save All**, and save into `~/lab2/net1/wireshark_export/` .

8. Open the exported image and record what it contains:

```
cd ~/lab2/net1/wireshark_export
file *
xdg-open <name-of-the-png-file>
```

9. Hash your evidence so you can prove later that it was not altered:

```
sha256sum ~/lab2/net1/net1.pcap ~/lab2/net1/wireshark_export/*
```

NetworkMiner: let the tool carve the evidence for you

10. Load the capture. In NetworkMiner choose `File > Open` and select `~/lab2/net1/net1.pcap` .

11. Work through these tabs and note what each one shows you:

- **Hosts:** expand the host. What operating system, open ports and web server details does NetworkMiner infer?
- **Files:** NetworkMiner extracts every transferred file automatically. Note each file name, size, source and destination. Right-click a file and choose `Open folder` to see where it is stored on disk.
- **Images:** a thumbnail of every picture in the traffic. The stolen content should be visible here without opening anything.
- **Parameters:** the form fields submitted with the upload, such as `fileToUpload` and `submit` .

12. Compare with Wireshark. Do the file names and sizes match what you exported in step 7? Which tool got you to the evidence faster?

ntopng: look at the traffic as flows on a dashboard

13. Run ntopng against the capture file:

```
cd ~/lab2/net1
# NOTE: FOR AMD ARCHITECTURE USE:
docker run -it --rm -p 3000:3000 -v $(pwd)/ntopng.license:/etc/ntopng.license:ro --
net=host docker.io/ntop/ntopng:latest -i /pcaps/net1.pcap
# NOTE: FOR ARM (M-Series Macbooks) USE:
docker run -it --rm -p 3000:3000 -v $(pwd)/ntopng.license:/etc/ntopng.license:ro --
net=host ntop/ntopng_arm64.dev:latest -i /pcaps/net1.pcap
```

> ntopng will run in Community mode, which is all this lab needs. Leave this terminal open while you work.

14. Open the web interface at `http://localhost:3000` in Firefox. The first login is `admin` / `admin`, and you will be asked to set a new password.

15. Explore and record:

- **Hosts:** which hosts were seen, and which one sent or received the most bytes?
- **Flows:** list the flows. Which application protocol does ntopng assign to them (look for `HTTP`), and which flow is the largest?
- **Interface view** (click the `net1.pcap` interface): note the total traffic volume and the protocol breakdown.

> Tip: if the live dashboard looks empty, that is normal. The capture is from the past, so use the Hosts and Flows pages instead of the real-time charts.

16. Stop ntopng with `Ctrl + C` in the terminal.

17. Reflect: ntopng cannot show you the stolen picture, so why might this tool be useful?

Part 2 - `net2.pcap`

The IR team believes the threat actor changed his technique for this capture. Repeat the same three-tool process, and pay close attention to what is **different** this time.

Wireshark

1. Open `~/lab2/net2/net2.pcap`. Look at `Statistics > Protocol Hierarchy` and `Statistics > Conversations` again.

Compare with `net1.pcap`:

- Which new protocols appear (look for `DNS`, `TLS` and `QUIC`)?
- Is there a new IP address outside the local network? How much data came from it?

2. Find out where that outside traffic went. Apply this filter:

```
dns || tls.handshake.extensions_server_name
```

Which domain names were looked up, and which server name appears in the TLS `Client Hello`? One of them is browser background noise and one of them is not. Which is which?

3. Line up the timeline. Apply this filter:

```
http || tls.handshake.type == 1
```

Write down, in order, when the outside download happened and when the upload to `upload.php` happened. How many seconds apart were they?

> TLS is encrypted, so you cannot see what was downloaded from the outside server. Compare the size of that download (from `Statistics > Conversations`) with the size of the file uploaded a few seconds later.

4. Follow the upload and retrieval (`POST /upload.php` and `GET /uploads/...`) with `Follow > HTTP Stream`, exactly as in Part 1. What is the file name this time?

5. Export the HTTP objects to `~/lab2/net2/wireshark_export/` and open the image. At first glance, does it look like company secrets?

NetworkMiner

6. Load `~/lab2/net2/net2.pcap` (use `File > Clear GUI` first) and review the Hosts, Files, Images and Parameters tabs.

- How many hosts appear now compared with `net1.pcap`?
- Which files could NetworkMiner extract, and which traffic could it **not** extract anything from? Why?

ntopng

7. Run ntopng against the second capture:

```
cd ~/lab2/net2
# NOTE: FOR AMD ARCHITECTURE USE:
docker run -it --rm -p 3000:3000 -v $(pwd)/ntopng.license:/etc/ntopng.license:ro --
net=host docker.io/ntop/ntopng:latest -i /pcaps/net2.pcap
# NOTE: FOR ARM (M-Series Macbooks) USE:
docker run -it --rm -p 3000:3000 -v $(pwd)/ntopng.license:/etc/ntopng.license:ro --
net=host ntop/ntopng_arm64.dev:latest -i /pcaps/net2.pcap
```

8. In the Hosts and Flows pages, find the outside host, the application ntopng labels it with, and how many bytes it sent. In this capture, the dashboard view makes the outside download stand out right away. Stop ntopng with `Ctrl + C` when done.

Uncover the hidden message

The image the threat actor uploaded looks innocent. The IR team suspects he used **steghide** to hide the real secret inside it. Let's test that.

> NOTE: You are free to use any tool, including an online stegsolver.
> (+5 extra credit "must show evidence of tool being used")

9. Try steghide first:

```
cd ~/lab2/net2/wireshark_export
steghide info <name-of-the-png-file>
```

Read the error carefully. steghide only works with JPEG, BMP, WAV and AU files, so it cannot have been used on this PNG as it is. That is a useful finding in itself: the actor either converted the picture after hiding the data, or hid the data a different way. Either way, the secret must be stored in the pixels themselves.

10. Look at the image's bit planes. Each pixel stores red, green and blue values. Changing only the lowest bits of those values is invisible to the eye, so hidden text often shows up when you look at each bit on its own. Save this script as `~/lab2/bitplanes.py`:

```
#!/usr/bin/env python3
# Usage: python3 bitplanes.py <image>
# Saves one black-and-white picture per color channel and bit (bits 0-3)
# into a folder called planes/ so you can flip through them.
import sys, os
import numpy as np
from PIL import Image

img = np.array(Image.open(sys.argv[1]).convert("RGB"))
os.makedirs("planes", exist_ok=True)
for name, ch in (("red", 0), ("green", 1), ("blue", 2)):
    for bit in range(4):
        plane = ((img[:, :, ch] >> bit) & 1) * 255
        out = f"planes/{name}_bit{bit}.png"
        Image.fromarray(plane.astype("uint8")).save(out)
        print("wrote", out)
```

11. Run it and flip through the results:

```
python3 ~/lab2/bitplanes.py <name-of-the-png-file>
xdg-open planes/
```

Open each `planes/*.png` file. Most will look like static. One of them will show a clear message near the top of the image. Record which channel and bit it was in, and what it says.

> Optional: the Java tool StegSolve does the same thing with arrow keys to cycle through the planes. Install it with `sudo apt install -y default-jre` and run `java -jar stegsolve.jar`.

12. Hash your evidence:

```
sha256sum ~/lab2/net2/net2.pcap ~/lab2/net2/wireshark_export/*
~/lab2/net2/wireshark_export/planes/*
```

Deliverables - What to Submit to Canvas:

1. Findings for the Client

Write **1 to 2 paragraphs for each capture** (four paragraphs at most in total) addressed to the client's executive team. Assume your readers are smart, busy and **not technical**.

Each write-up must answer:

1. **What was the threat actor doing?**

2. **How did he get the secrets out of the network?**
3. **How did he try to hide what he was doing?** (For `net1.pcap`, it is fair to say whether he tried to hide it at all.)

For `net2.pcap`, also explain **what changed** compared with the first capture and why that change matters to the business.

Writing guidelines

- Lead with the impact: what was taken and how serious it is.
- Use plain words. Write "a web upload form" rather than "an HTTP POST with multipart/form-data". If a technical term is unavoidable, explain it in half a sentence.
- Be specific and factual: name the file, the time and the destination. Separate what the evidence **shows** from what you **suspect**.
- Leave out tool names, packet numbers and filter syntax. Put those in an optional appendix if you want to show your work.
- End each write-up with one sentence on what the client should do next.

Answers to Questions in Part 1 and Part 2

Grading

Criteria	Points
Accurately describes what the actor did in each capture	30
Clearly explains how the data left the network	20
Explains how the actor tried to conceal the activity, and what changed in `net2.pcap`	20
Written for executives: clear, plain language, no unnecessary jargon	15
Answers to Questions in Part 1 and Part 2	5
Total	**100**